AGENTS + AI

AGENTS VAULT

Five under-surfaced tools for building, running, and watching AI agents — each with a ready-to-paste prompt.

TUNED FOR LINUX · bash

JUNE 2026 · 5 PICKS · EN / AR

> smolagents > Agency Swarm > microsandbox > Letta > Laminar

curated by Hamzah Al-Ramli · goodnbad.info

Welcome to the Vault.

Everyone's talking about AI agents; almost nobody shows you the tooling that actually makes them work. This week is the agent stack the hype skips — the framework you build on, the orchestration that keeps it sane, the sandbox that keeps it safe, the memory that makes it remember, and the lens that lets you see what it did. Real repos, real links, and a prompt you paste straight in to start.

▸ HOW TO USE EACH PICK

1. Read the hook — decide in 5 seconds if it's for you.
2. Open the GitHub link.
3. Copy the **>_ DROP THIS INTO YOUR AI** prompt into Claude, ChatGPT, or Codex.
4. Let it walk you through setup & first use. No guesswork.

▸ YOUR BUILD · LINUX Commands are written for bash — use your distro's package manager (apt / dnf / pacman). Run everything in a scoped, authorized environment.

الكل يتكلم عن الـ AI agents، بس قليل اللي يوريك الأدوات اللي تخليها تشتغل فعلاً. هالأسبوع جمعنا لك stack كامل للـ agents: الإطار اللي تبني عليه، الطبقة اللي تنظّم وكلائك، الصندوق اللي يأمّن الكود، الذاكرة اللي تخلّي الـ agent يتذكر، والعدسة اللي توريك وش سوّى بالضبط. ريبوهات حقيقية، روابط حقيقية، وبرومبت جاهز تنسخه في Claude أو ChatGPT أو Codex وتبدأ على طول.

IN THIS VAULT

AI AGENTS / FRAMEWORK · HUGGING FACE

smolagents

01

★ ~28k · Hugging Face official · verdict BOTH

Most agent frameworks make the model spit JSON for every step. smolagents lets the agent write actual Python instead — the whole engine fits in about a thousand lines, and it runs any model you want.

> أغلب أطر الـ agents تخلّي النموذج يطلّع JSON لكل خطوة. smolagents يخلّي الـ agent يكتب بايثون حقيقي بدالها — والمحرك كله بحدود ألف سطر، ويشتغل على أي نموذج تبيه، محلي أو سحابي.

WHAT IT IS

smolagents is Hugging Face's minimalist library for agents that act by writing and running code, not by emitting tool-call JSON. The core logic is tiny on purpose, so you can actually read it and understand what your agent is doing under the hood. It's model-agnostic — local transformers, Ollama, or any cloud LLM through LiteLLM — and it runs the generated code inside sandboxes like E2B or Docker so it stays contained. You can pull and share tools straight from the Hub.

WHY IT MATTERS FOR YOU

When you're starting out, the big frameworks bury you in abstraction before you've built anything. This is the opposite: small enough to grasp in an afternoon, powerful enough to ship real work. Code-as-action also tends to need fewer steps than JSON tool-calling, so your agent gets to the answer faster.

repo > github.com/huggingface/smolagents

>_ DROP THIS INTO YOUR AI · Linux / bash

```
Set up Hugging Face smolagents (github.com/huggingface/smolagents) to build my first
code-writing agent. Give me the exact steps: install it, point it at a model (show me
both a free Hub model and an Anthropic/OpenAI option via LiteLLM), and build a small
CodeAgent that can search the web and run Python. Explain how the code-as-action loop
differs from normal tool-calling as you go.
```

AI AGENTS / ORCHESTRATION · PYDANTIC-TYPED

Agency Swarm

02

★ ~4.5k · extends OpenAI Agents SDK · verdict BOTH

One agent is easy. Five agents talking to each other without going off the rails is the hard part. Agency Swarm is the reliability layer — typed tools, clean hand-offs, deterministic behavior.

agent واحد سهل. خمس agents يتكلمون مع بعض بدون ما يطيحون — هذا اللي صعب. Agency Swarm هو طبقة الموثوقية: أدوات مكتوبة بأنواع صارمة، تسليم نظيف بين الوكلاء، وسلوك متوقّع.

WHAT IT IS

Agency Swarm is a multi-agent orchestration framework built on top of the OpenAI Agents SDK, focused on making swarms behave predictably instead of chaotically. You define roles — a CEO agent, a developer, an assistant — each with its own instructions and tools, and wire up exactly who can talk to whom. Every tool is a Pydantic model, so arguments get validated automatically and you get real type safety and error handling. It's designed for production deployment from the start, not just demos.

WHY IT MATTERS FOR YOU

Multi-agent setups fail quietly: an agent passes garbage to another, nobody validates it, and the whole chain drifts. This framework's typed tools and explicit communication flows are exactly what stops that. Think of it as FastAPI for agents — opinionated, well-typed, and built to actually go live.

repo > github.com/VRSEN/agency-swarm

>_ DROP THIS INTO YOUR AI · Linux / bash

```
Help me build a reliable multi-agent system with Agency Swarm (github.com/VRSEN/agency-swarm). Install it, then scaffold a small agency: a manager agent plus two worker agents with defined communication flows. Show me how to write a tool as a typed Pydantic model with validation, and explain how the hand-offs and roles keep the swarm deterministic instead of chaotic.
```

AI AGENTS / SECURITY · SELF-HOSTED

microsandbox

03

★ ~6.6k · Firecracker-style isolation, self-hosted · verdict BOTH

Your agent just wrote code and wants to run it. On your machine. With your files. microsandbox boots a real microVM in milliseconds so untrusted agent code runs fully isolated — no cloud bill, no trust.

> الـ agent كتب كود وودّه يشغّله — على جهازك وبملفاتك. microsandbox يقلّع لك microVM حقيقي بأجزاء من الثانية، فالكود اللي ما تثق فيه يشتغل معزول تمامًا — بدون فاتورة سحابة، وبدون ما تسلّم جهازك.

WHAT IT IS

microsandbox runs untrusted code inside lightweight, fast-booting virtual machines using familiar container-style workflows and standard OCI images. Unlike plain Docker, it gives each session real VM-level isolation — its own kernel — so agent-generated code can't reach your host. It's built to be self-hosted and local: you run it on your own machine instead of shipping code off to a managed cloud sandbox. Boot times are fast enough to feel interactive, not batch.

WHY IT MATTERS FOR YOU

The single scariest part of agentic coding is letting a model execute what it just wrote. Most people either run it raw (dangerous) or pay per-second for a cloud sandbox. This is the third option: strong isolation, on your own hardware, free. It's the safety layer that makes picks 01 and 02 actually safe to let loose.

repo > github.com/microsandbox/microsandbox

>_ DROP THIS INTO YOUR AI · Linux / bash

```
Set up microsandbox (github.com/microsandbox/microsandbox) so my AI agent can run its own generated code safely on my machine. Give me the install and the exact steps to spin up an isolated sandbox, run a Python snippet inside it, and confirm it can't touch my host files. Then show me how to call it from an agent loop so every code action gets sandboxed automatically.
```

AI AGENTS / MEMORY · THE MEMGPT TEAM

Letta

04

★ ~23k · the team behind MemGPT · verdict BOTH

Your agent forgets everything the moment the context window fills up. Letta treats the context like virtual memory — core, recall, and archival tiers — so your agent actually remembers and improves over time.

الـ agent ينسى كل شي أول ما يمتلئ السياق. Letta يتعامل مع السياق كأنه ذاكرة افتراضية — طبقة أساسية، وطبقة استرجاع، وطبقة أرشيف — فالـ agent يتذكّر فعلاً ويتحسّن مع الوقت.

WHAT IT IS

Letta is the open-source runtime from the team that created MemGPT, built around the idea that an agent's context should work like a computer's memory hierarchy. It manages three tiers automatically: core memory inside the live context, recall memory for searchable conversation history, and archival memory for long-term storage — paging facts in and out as needed. Letta runs the whole agent loop, tool execution, and memory management for you, with support for skills, subagents, and continual learning. It ships as both a local CLI and an API you can build on.

WHY IT MATTERS FOR YOU

Memory is the difference between a chatbot that resets every session and an assistant that genuinely knows your project after a month. Most 'memory' libraries just bolt a vector store on the side; Letta makes memory the core of how the agent runs. If you're building something that should remember and get better, this is the layer.

repo > github.com/letta-ai/letta

>_ DROP THIS INTO YOUR AI · Linux / bash

```
Set up Letta (github.com/letta-ai/letta), the runtime from the MemGPT team, so my agent
has real long-term memory. Walk me through installing the server, creating a stateful
agent via the CLI, and having a conversation where it remembers facts across sessions.
Explain the core / recall / archival memory tiers and how Letta decides what to page in
and out.
```

AI AGENTS / TRACING · OPEN SOURCE

Laminar

05

★ ~3k · OpenTelemetry-native · self-hostable · verdict BOTH

When an agent does something dumb, 'it just decided to' isn't a debug log. Laminar traces every step, tool call, and decision — and lets you run SQL over your agent's behavior to find exactly where it went wrong.

لمّا الـ agent يسوّي شي غبي، 'قرّر كذا' مش تقرير أخطاء. Laminar يسجّل كل خطوة وكل استدعاء أداة وكل قرار — ويخليك تشغّل SQL على سلوك الـ agent عشان تلقى بالضبط وين غلط.

WHAT IT IS

Laminar is an open-source observability platform built specifically for AI agents, not generic apps. It's OpenTelemetry-native, so it traces every LLM call, tool invocation, and step in your agent's run as structured spans you can actually inspect. Beyond raw tracing it gives you a transcript view of long agent sessions, evaluations, dashboards, and the ability to query your traces with SQL to spot patterns and failures. You can self-host the whole thing.

WHY IT MATTERS FOR YOU

The first time an agent burns ten steps on the wrong path, you'll wish you could rewind and see why. Without tracing, agents are black boxes — you tweak prompts blind. Laminar is the lens: build with 01, orchestrate with 02, sandbox with 03, remember with 04, and this is how you finally see what all of it actually did.

repo > github.com/lmnr-ai/lmnr

>_ DROP THIS INTO YOUR AI · Linux / bash

```
Set up Laminar (github.com/lmnr-ai/lmnr) to trace and debug my AI agent. Show me how to
self-host it or use the cloud, instrument my agent with the SDK so every LLM call and
tool step is captured, and open the transcript view for a full run. Then give me an
example SQL query over the traces to find where the agent wasted steps or failed.
```

That's the Vault for this week.

Five tools that stack into one real agent pipeline — build, orchestrate, sandbox, remember, observe. Pick one layer this week and wire it in; that's the whole point. Real infrastructure you run, not another thread to scroll.

Next week: more under-surfaced agent tooling — eval harnesses, browser agents, and the deployment layer most people skip.

Found one of these useful? Forward this to one person who'd get it. That's how the Vault grows.

خمس أدوات تتركّب فوق بعض لتعطيك pipeline حقيقي للـ agents: ابنِ، نظّم، أمّن، تذكّر، راقب. خذ طبقة وحدة هالأسبوع وركّبها — هذا كل الهدف. بنية تحتية تشغّلها فعلاً، مش محتوى تتصفحه بس. عجبتك وحدة؟ ابعثها لشخص يستفيد. هكذا تكبر الخزينة.